FORMATIVE ASSIGNMENT

PermitWatch Rwanda

- **Description of the concept of their Mission;**

My mission is to be an entrepreneur in the technology sector, with a primary focus on building a state-of-the-art facility that develops software, hardware, and systems specifically for businesses and banking in Africa. I am driven by the reality that most African enterprises rely on foreign technology—from the United States, Europe, and China—that is often difficult to use and poorly adapted to our local contexts. Observing how African users struggle with these imported systems, I recognized both the challenge and the enormous opportunity for Africans to design, build, and implement technology that truly meets our needs. My goal is to capture this opportunity by creating African-made solutions that empower businesses, streamline banking, and strengthen our technological independence. This is the mission which drives me and helps through my journey since starting and getting introduced to the mission at ALU.

- **Problem statement:**

The service with the highest bribe in Rwanda is obtaining a construction permit from the local government, at 22.90% of total service-related corruption (Transparency International Rwanda, 2025). This affects construction project owners, who get to experience unfairness, inequality, and injustice. These problems slow down the construction sectors in Rwanda and limit equal access to public services in Rwanda.

- **Software development model**

Agile with prototype models, since this product won't have certain access, Application Programming Interface(API). To make it happen and get a working product, an agile approach will be used, as it’s time-efficient and ideal for short-term development. But the prototype is their cause, the product will be working, and the required API will be there as a simulation of them, generating data through code, the product will be available at that time, will be a fully working prototype with both frontend, backend, and database.

- **The hypothesis of their solution**

By the time the product is available, 100% of injustice and corruption involved in building permits in Rwanda will be detected and seen by the Office of the Ombudsman, whose responsibilities involve fighting against corruption and injustice. As the purpose of the application is to detect that action will be taken by law enforcement, and they have data and evidence, action will be taken against every case detected to reduce corruption and injustice involved in building permits to 99%.

1. Software Requirements Specification (SRS)

Title: PermitWatch Rwanda

Prepared by Ishami Irené

African Leadership University

Wednesday, 27 May 2026

1.1. Table of content

1.2. Revision History

Name	Date	Reason For Changes	Version
Ishami Irené	27/5/2026	The starting document	1
Ishami Irené	25/6/2026	Refined the document based on comment from facilitator (Requirements grammar refined, hypothesis and SDLC added to document)	2.0
Ishami Irené	25/6/2026	Added UML diagrams, at glossary B for analysis	2.1

2. Introduction

2.1. Purpose

This software Requirements specification (SRS) document describes the requirements for **PermitWatch version 1.0**, a technological platform designed to improve transparency and equality in construction permit approval by local governments in charge.

The system aims to keep law enforcement in charge of fighting against corruption and injustice, specifically the Office of Ombudsman, to keep track of construction permit requests, monitor permit approval timelines, alert permit approvals of high probability of corruption or injustice by checking the overdelayed permit approvals, and the First-in First-out (FIFO) broken, when the approval gets way before others.

This Software Requirement Specification (SRS) document covers core functionalities of the PermitWatch System, including:

- User registration and authentication
- Permit application tracking
- Report the alerted permits.
- Administrative dashboard for tracking the permits

The SRS mainly focuses on web-based applications. This will be fully government-based and used by government agencies. External government systems will be involved for this PermitWatch Version 1.0. This version is used exclusively by government agencies, mainly the Office of the Ombudsman.

2.2. Document Conventions

This SRS document follows Academic writing convention standards (Rollins College Tutoring & Writing Center, n.d). Keep one main idea per paragraph, and indent the first line. Use 1.0 (single) or 1.5 spacing. Use bolding, bullet points, and numbered lists to break up dense blocks of text.

The headings are listed in the hierarchy. and bold texts on important terms, as well as italic for quoted words.

The functional and non-functional requirements are listed separately in the table, and written in clear, testable statements. Each requirement has its own priority level, not by inheriting the priority level, and with a description.

The bullets were used in case needed and appropriately.

2.3. Intended Audience and Reading Suggestions

The intended readers of this SRS document include the Office of the Ombudsman Rwanda, Transparency International Rwanda, relevant government institutions, developers, testers, and other stakeholders involved in anti-corruption and justice-related activities in Rwanda.

2.4. Product Scope

The business strategy for this project is to develop and deploy the complete software solution for relevant government and anti-corruption institutions, after which the system will be licensed or adopted by those organizations. Continuous technical support, updates, and system maintenance services will be provided through a maintenance agreement or service fee structure.

3. Overall Description

3.1. Product Perspective

3.1.1. 2.1.1. Product Background and Origin

The background is well explained through the problem statement, which is this following: "*The service with the highest bribe in Rwanda is obtaining a construction permit from the local government, at 22.90% of total service-related corruption (Transparency International Rwanda, 2025). This affects construction project owners, who get to experience unfairness, inequality, and injustice. These problems slow down the construction sectors in Rwanda and limit equal access to public services in Rwanda.*"

The **Permitwatch** system is designed to provide law enforcement with the capability to track and monitor construction permit applications in real time and get alerted in case any of the reasons in section 1.1, paragraph 2, mention is up to them to act on it immediately.

This is the first system only available, the first version, not an upgrade to an available system that might be working and solving all available problems, even though it is the first system. But it will depend on the currently available and working Kubaka system, which is being used by the Local government and the Rwanda Housing Authority in application processes at the moment.

3.1.2. Relations with other systems

PermitWatch will directly interact with the available Kubaka system. It is fully dependent on Kubaka, which the Local government and the Rwanda Housing Authority currently use in application processes. PermitWatch will not change anything on the Kubaka system. The connection will simply access information on every application and approval made across the country.

The Kubaka system APIs will supply PermitWatch with real-time data regarding all application and approval activities. The information transmitted via the APIs includes project owners' details (such as name, ID, phone number, and email), land data (UPI and location), project specifics (building level and requested permit), and engineers' contact details. The APIs will also provide links to associated application files. This structured data flow ensures tracking of all submitted applications and approved permits across the country, supporting transparency and reducing opportunities for corruption or injustice.

3.1.3. Interfaces and connections

The following are the connections in the system:

- **users and system:** users will be capable of visualizing the data and filtering out the data based on location, time, and approval response.
- **Frontend and backend:** Frontend will be getting the calling backend to get updated regarding the information available.
- **Backend and database:** backend will be adding and calling the data from the database, but can't edit the data in the database by either adding or changing the data.
- **External services/APIs**: Kubaka system would be providing the information, that is why the backend should not edit the data from the Kubaka system, which will be the one to decide if there is corruption or injustice.

3.2. Product Functions

The main function of the PermitWatch system is to collect and analyze permit-related data in order to identify possible cases of corruption or unfair practices for the responsible authorities for further action.

The system will provide the following major functionalities:

- User management
- User authentication and login
- Interactive dashboard
- Permit data tracking and monitoring
- Corruption and anomaly detection

3.3. User Classes and Characteristics

This will be strict regarding users, since it will have a certain level of access to personal data, and its function will be to fight corruption and injustice. When either the chief ombudsman or the deputy ombudsman grants someone access and login details, that person begins tracking, monitoring, and alerting the law enforcement in charge; that person is the non-technical user..

User class	Responsibility	Access level	Technical skills	priority
Chief ombudsman/ Deputy ombudsman	1. Create user and give them log in credential 2. Access the current user 3. Track and monitor data	Have access to admin level access	No technical skills required	Non-negotiable
Non-technical user (person given access by chief ombudsman/ deputy ombudsman)	1. Track and monitor permit data 2. Alert the laws enforcement agencies in case of corruption or injustice	Have access to data and contact of law enforcement agencies	No technical skills required	Low priority (can go way easily)
System maintenance Technician	1. Maintain the system and refine the system to meet the users wants	Technical admin level	Only technical skills required	High priority

3.4. Operating Environment

The system will be deployed in the cloud, and the user will be able to use it effectively and efficiently. On the browser, no big technical environment is needed; all the user has is a browser and an internet connection to access and utilize the system.

3.5. Design and Implementation Constraints

The system development is subject to several design and implementation constraints. The system will be built using the Python programming language and the Flask framework to allow a large number of technical users capable of maintaining the system. Since it is browser-based, this means there are no hardware constraints.

The system must integrate the Kubaka system data through REST APIs, which might be the hardest part to get to. For the system demo to work, the REST APIs will be used with simulation data generation. All communication will go through HTTPS, and the sensitive data will be encrypted under the institutional security standards.

Developers must follow coding conventions and documentation standards to make later maintenance easier. The system will be using the English language.

3.6. User Documentation

There will be two documentaries: a demo video and API documentation. Regarding the coding, everything will be in the README file. The demo video will be providing how to properly use the system for its importance and purpose. For API documentation, we will provide documentation for api endpoints and their usability, and how the integration was done.

3.7. Assumptions and Dependencies

The system will assume that the user has a stable internet connection and is using a supported internet browser, such as Google Chrome, Brave, or Firefox. The deployment server is expected to stay working all the time.

The project fully depends on the third-party Kubaka system, which handles data retrieval and processing through APIs, and stores information in a PostgreSQL database based on the information received and actions taken.

It is assumed that the selected frameworks and APIs will continue during the project life cycle, development, and upgrading. Any major changes to these systems may require changes in system design, implementation, or overall changes.

4. External Interface Requirements

4.1. User Interfaces

The system shall provide the user interface for user authentication, dashboard, administrative functions, and reporting. All interfaces shall follow a layout and an easier navigation structure to improve usability and user experience.

The application shall use a responsive web-based interface with a mobile-first approach to make it easier for all device compatibility, like desktop, tablet, and mobile devices. Standard navigation shall appear on all pages, including home, dashboard, profile, and logout options.

The error messages and validation alerts will be provided on the screen, near the affected fields, for clear formatting. Confirmation dialogue shall appear in critical actions like deleting or updating certain fields.

4.2. Software Interfaces

The system will communicate with the Kubaka system with the available version, not a specific version at all. The purpose is to get the required information to fight against corruption and injustice happening in construction permits, where permits are requested and given on the Kubaka system. The communication will be through RESP APIs, the data sharing will be in JSON format, the data that will be exchanged will be project information, land information, project owner information, engineering companies involved information, or the engineer working on the project, time of construction permit request, and construction permit approvals. The file will not be shared; the link to download is the one to be shared with our system.

5. Requirement Specification

5.1. Stakeholder Requirements Specification

5.1.1. Functional Requirements

Req ID	Requirements	Description
FR 1	**User Authentication and Access Management**	The system shall provide secure authentication and user management capabilities.
FR 1.1	Chief Ombudsman/Deputy Ombudsman Initial Login	The system shall allow the Chief Ombudsman/Deputy Ombudsman to log in using administrator credentials provided during system deployment and shall require a password change upon first login.
FR 1.2	User Account Management	The system shall allow the Chief Ombudsman and Deputy Ombudsman to create, update, deactivate, and manage user accounts and access permissions.

FR 2	**User Login**	The system shall authenticate authorized users through Active Directory (AD).
FR 2.1	Capture AD Username	The system shall provide a field for users to enter their Active Directory username.
FR 2.2	Capture AD Password	The system shall provide a secure field for users to enter their Active Directory password.
FR 2.3	User Authentication	The system shall validate user credentials against the Active Directory service before granting access.
FR 3	**Dashboard**	The system shall provide an interactive dashboard for monitoring corruption and injustice cases.
FR 3.1	Dashboard Visualization	The system shall display case statistics, trends, alerts, and summaries through charts, graphs, and tables.
FR 4	**Corruption and Injustice Alerting**	The system shall generate alerts when predefined corruption or injustice indicators are detected.
FR 4.1	Alert Generation	The system shall generate alerts when predefined corruption or injustice indicators are detected.
FR 4.2	Notice law enforcement agencies	The system shall enable authorized users to notify relevant law enforcement agencies regarding identified cases.
FR 5	**Report Generation**	The system shall generate reports related to corruption and injustice cases.
FR 5.1	Generate Case Reports	The system shall allow authorized users to generate detailed reports for individual cases.
FR 5.2	Generate Summary Reports	The system shall generate summary reports showing case statistics, trends, alerts, and actions taken over a specified period

FR 5.3	Export Reports	The system shall allow users to export reports in PDF and Excel formats.
FR 5.4	Report Filtering	The system shall allow users to filter reports by date range, case status, department, and alert type.

5.2. Other Nonfunctional Requirements

5.2.1. Example of Non-Functional Requirements

Requirement Type	Req ID	Description
Security	NFR 1	The system shall ensure that only authenticated users created by authorized administrators can access the system.
Security	NFR 2	The system shall enforce role-based access control (RBAC) to restrict access according to user roles and responsibilities.
Usability	NFR 3	The system shall provide all user interfaces and messages in English.
Compatibility	NFR 4	The system shall function correctly on the latest versions of Chrome, Firefox, Edge, and Safari browsers.
Technology	NFR 5	The system shall be accessible from desktop computers, laptops, tablets, and Android devices.

Requirement Type	Req ID	Description
Availability	NFR 6	The system shall be available at least 99% of the time, excluding scheduled maintenance periods.
Performance	NFR 7	The system shall load the dashboard within 5 seconds under normal operating conditions.

6. Reference

1. Rollins College Tutoring & Writing Center. (n.d.). *Rollins College Tutoring & Writing Center*. Rollins College. https://lib.rollins.edu/olin/web-documents/twc/academic-writing-conventions.pdf
2. Transparency International Rwanda. (2025, December 3). *Rwanda Bribery Index 2025* (16th Edition ed.) [Assesses the experiences and perceptions of bribery as a specific form of corruption in Rwanda] [Website / Report]. Transparency International Rwanda. Retrieved May 24, 2026, from https://www.tirwanda.org/IMG/pdf/rwanda_bribery_index_rbi_2025.pdf

7. Appendix

<Define any other requirements not covered elsewhere in the SRS. This might include database requirements, internationalization requirements, legal requirements, reuse objectives for the project, and so on. Add any new sections that are pertinent to the project.>

7.1. Appendix A: Glossary

<Define all the terms necessary to properly interpret the SRS, including acronyms and abbreviations. You may wish to build a separate glossary that spans multiple projects or the entire organization, and just include terms specific to a single project in each SRS.>

Acronym	**Description**
ALU	African Leadership University
API	Application Programming Language
SRS	Software Requirement Specification
FIFO	First In First Out
ID	Identification
REST API	Representational State Transfer Application Programming Interface
HTTPs	Hypertext Transfer Protocol Secure
AD	Active Directory
UML	Unified Modelling Language
DNS	Domain Name System
SPF	Sun Protection Factor
DKIM	DomainKeys Identified Mail
DMARC	Domain-based Message Authentication, Reporting, and Conformance

WAF	Web Application Firewall

7.2. Appendix B: Analysis Models

<Optionally, include any pertinent analysis models, such as data flow diagrams, class diagrams, state-transition diagrams, or entity-relationship diagrams.>

This part is for Unified Modelling Language (UML) for this system which includes 2 structural diagrams(class and deployment diagrams) and 2 behavioral diagrams(use case and activity diagrams) for better understanding of the system.

7.2.1. Class diagram

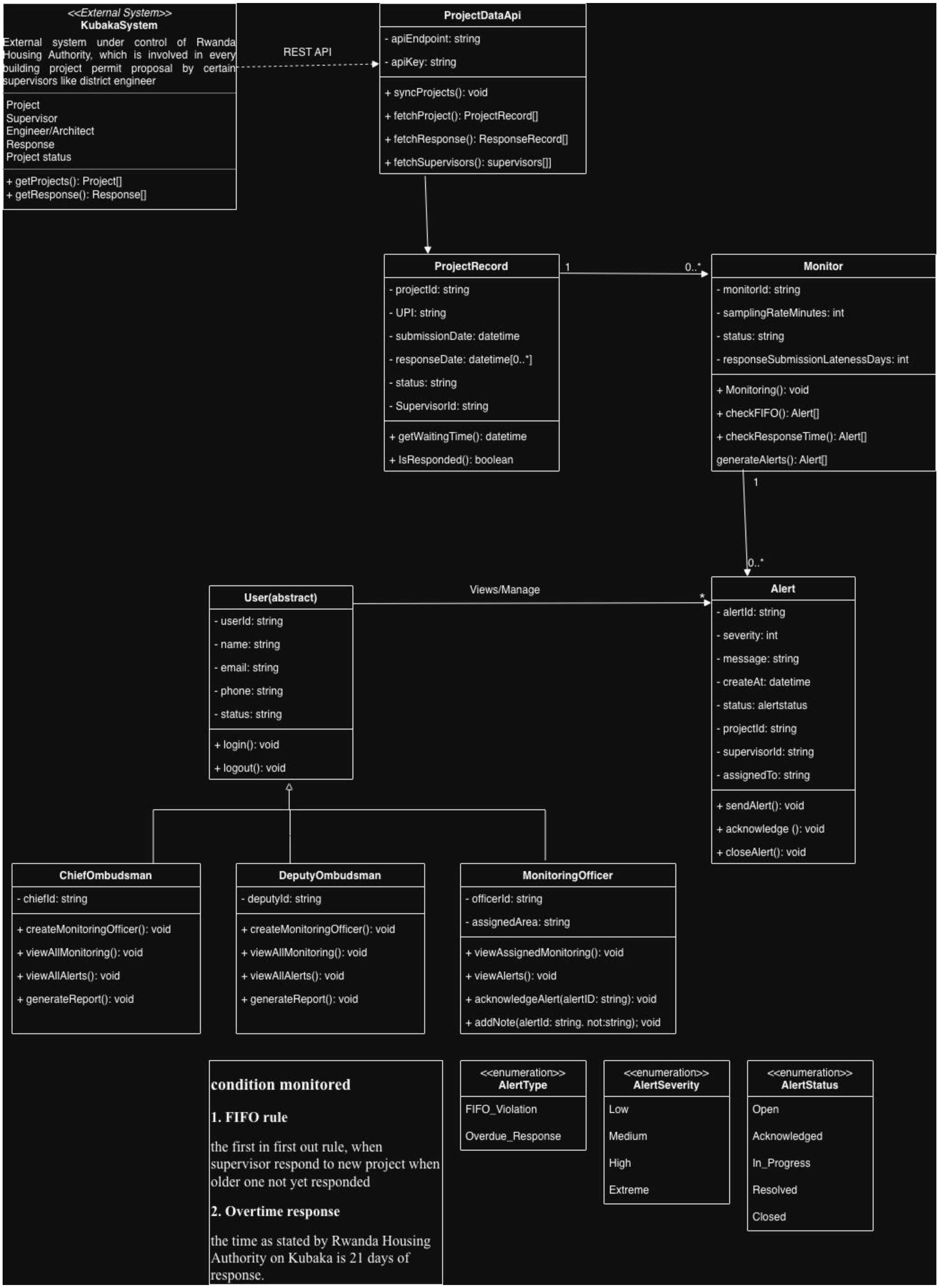
<<External System>>
KubakaSystem
External system under control of Rwanda Housing Authority, which is involved in every building project permit proposal by certain supervisors like district engineer
Project
Supervisor
Engineer/Architect
Response
Project status
+ getProjects(): Project[]
+ getResponse(): Response[]
REST API
ProjectDataApi
- apiEndpoint: string
- apiKey: string
+ syncProjects(): void
+ fetchProject(): ProjectRecord[]
+ fetchResponse(): ResponseRecord[]
+ fetchSupervisors(): supervisors[]]
ProjectRecord
- projectId: string
- UPI: string
- submissionDate: datetime
- responseDate: datetime[0..*]
- status: string
- SupervisorId: string
+ getWaitingTime(): datetime
+ IsResponded(): boolean
1
0..*
Monitor
- monitorId: string
- samplingRateMinutes: int
- status: string
- responseSubmissionLatenessDays: int
+ Monitoring(): void
+ checkFIFO(): Alert[]
+ checkResponseTime(): Alert[]
generateAlerts(): Alert[]
1
0..*
Views/Manage
*
User(abstract)
- userId: string
- name: string
- email: string
- phone: string
- status: string
+ login(): void
+ logout(): void
Alert
- alertId: string
- severity: int
- message: string
- createAt: datetime
- status: alertstatus
- projectId: string
- supervisorId: string
- assignedTo: string
+ sendAlert(): void
+ acknowledge (): void
+ closeAlert(): void
ChiefOmbudsman
- chiefId: string
+ createMonitoringOfficer(): void
+ viewAllMonitoring(): void
+ viewAllAlerts(): void
+ generateReport(): void
DeputyOmbudsman
- deputyId: string
+ createMonitoringOfficer(): void
+ viewAllMonitoring(): void
+ viewAllAlerts(): void
+ generateReport(): void
MonitoringOfficer
- officerId: string
- assignedArea: string
+ viewAssignedMonitoring(): void
+ viewAlerts(): void
+ acknowledgeAlert(alertID: string): void
+ addNote(alertId: string. not:string); void
condition monitored
1. FIFO rule
the first in first out rule, when supervisor respond to new project when older one not yet responded
2. Overtime response
the time as stated by Rwanda Housing Authority on Kubaka is 21 days of response.
<<enumeration>>
AlertType
FIFO_Violation
Overdue_Response
<<enumeration>>
AlertSeverity
Low
Medium
High
Extreme
<<enumeration>>
AlertStatus
Open
Acknowledged
In_Progress
Resolved
Closed

Figure 7.1: class diagram for Permit watch

I. **Purpose**

The class diagram presents the static structure of the system by showing classes, attributes, methods, and relationships.

II. **Description**

The system consists of the following major classes:

User (Abstract Class)

Represents all system users and contains common attributes such as:

- User ID
- Name
- Email
- Phone Number
- Status

Common operations include:

- Login()
- Logout()

Chief Ombudsman

Inherits from User and is responsible for:

- Creating monitoring officers
- Viewing monitoring activities
- Viewing alerts
- Generating reports

Deputy Ombudsman

Inherits from User and performs administrative functions similar to the Chief Ombudsman.

Monitoring Officer

Responsible for:

- Viewing assigned alerts
- Acknowledging alerts
- Adding notes and actions
- Monitoring assigned projects

ProjectDataApi

Handles communication with the Kubaka System through REST APIs.

Functions include:

- Synchronizing projects
- Fetching project data
- Retrieving responses
- Obtaining supervisor information

ProjectRecord

Stores project information such as:

- Project ID
- UPI
- Submission Date
- Response Date
- Status
- Supervisor ID

Monitor box

This is the table containing rules used throughout the process and decision making if either one needs Monitoring officer attention or not.

Monitoring rules include:

1. **FIFO Violation**
   - This is a case where a newer project is responded to way before older projects are responded to.
2. **Overdue Response**
   - A project spent over 21 days not responded, where 21 days are the time for response, but this rule can change with new days put on by people in charge which is Rwanda Housing Authority.

Alert

Represents detected violations and contains:

- Alert ID
- Severity
- Message
- Creation Date
- Status
- Assigned Officer

The Alert class supports operations such as:

- Send Alert
- Acknowledge Alert
- Close Alert

Relationships

- ProjectDataApi retrieves data from Kubaka System.
- ProjectRecord objects are analyzed by Monitor.
- Monitor generates multiple Alerts.
- Monitoring Officers manage assigned Alerts.
- The Chief Ombudsman and Deputy Ombudsman supervise monitoring activities.

III. Contribution

The class diagram serves as the foundation for software implementation by defining system entities, their responsibilities, and relationships.

7.2.2. Deployment diagram

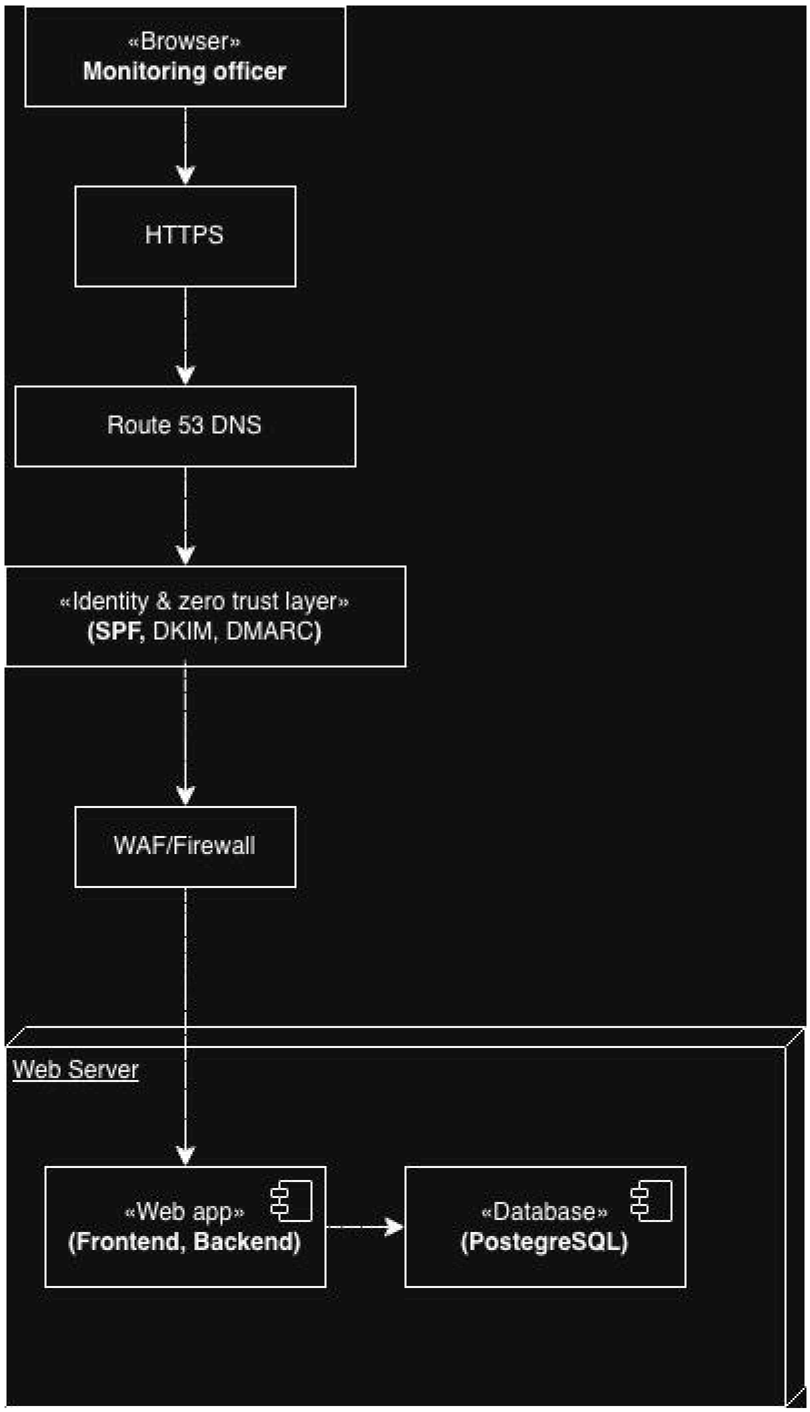

Figure 7.2: deployment diagram

I. Purpose

The deployment diagram illustrates the physical architecture of the system and how software components are deployed across infrastructure nodes. This give insight regarding what kind of cloud is either public or private.

II. Description

The architecture consists of the following components:

Client Layer

The Monitoring Officer accesses the system through a web browser.

Network Layer

User requests pass through:

- HTTPS secure communication
- Route 53 DNS service
- Identity and Zero Trust Layer (SPF, DKIM, DMARC)
- Web Application Firewall (WAF)

These layers provide secure routing, authentication, and protection against malicious traffic.

Application Layer

The Web Server hosts:

- Frontend application
- Backend application

The backend processes user requests, executes monitoring logic, and communicates with the database.

Data Layer

A PostgreSQL database stores:

- User information

- Project records
- Monitoring results
- Alerts
- Reports

III. Contribution

The deployment diagram demonstrates how the system components interact within a secure environment and supports scalability, reliability, and secure access to monitoring services.

7.2.3. Use case diagram

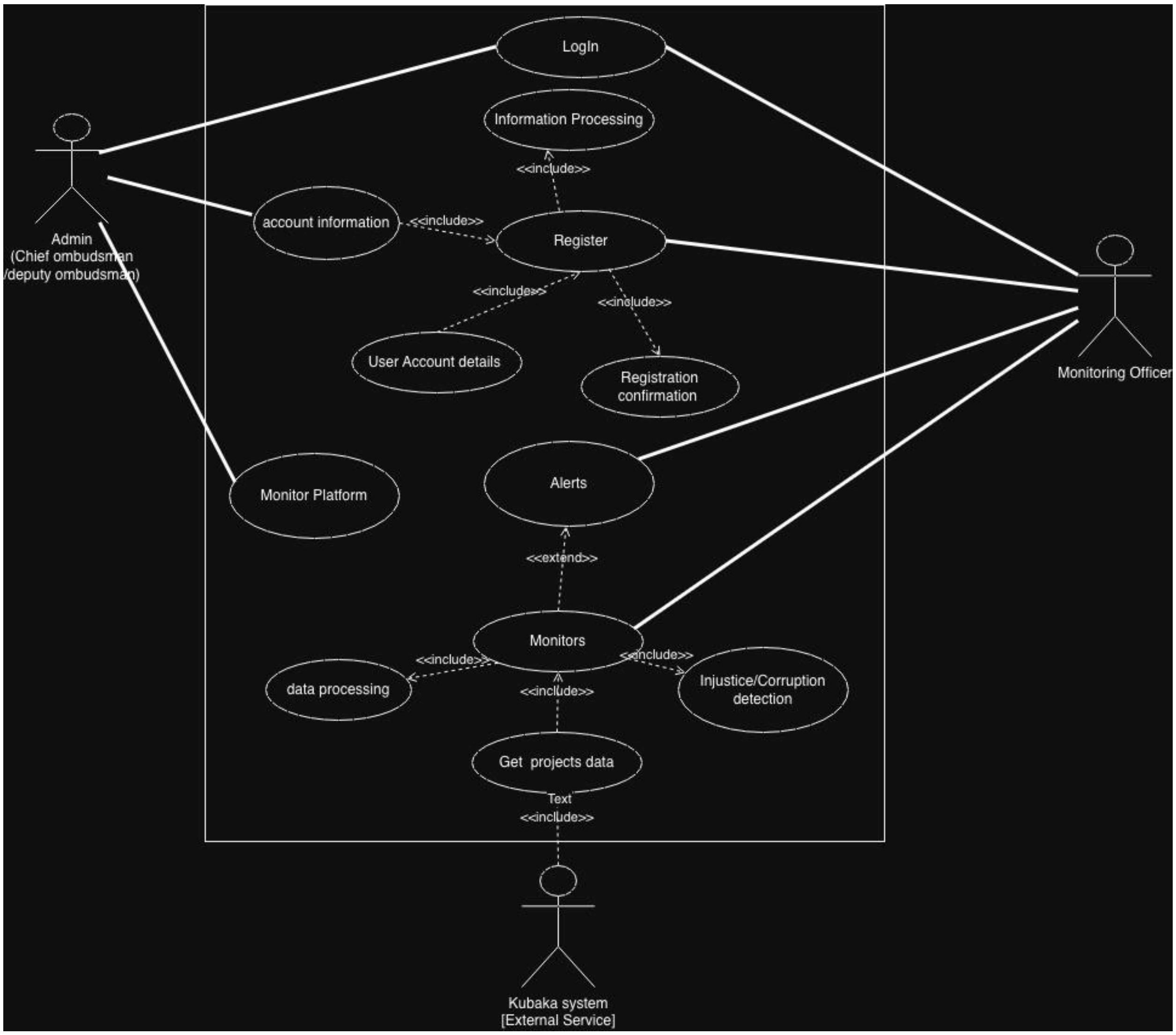

Figure 7.3: use case diagram

I. Purpose

The use case diagram illustrates the interactions between system users and the monitoring platform. It identifies the main actors and the services they can access within the system.

II. Description

The system has three primary actors:

- **Admin (Chief Ombudsman / Deputy Ombudsman)** who manages the platform and creates monitoring officer accounts.
- **Monitoring Officer** who reviews alerts, monitors project compliance, and updates alert statuses.
- **Kubaka System** which serves as an external source of project data through APIs.

The major use cases include:

- Login
- Register users
- Manage account information
- Monitor projects
- Process project data
- Detect violations
- Generate alerts
- Retrieve project information from Kubaka System

The diagram shows that monitoring activities depend on project data received from the external Kubaka System. Whenever violations are detected, the system generates notifications for monitoring officers.

III. Contribution

This diagram provides a high-level view of the functional requirements and defines how different users interact with the system.

7.2.4. Activity flow diagram

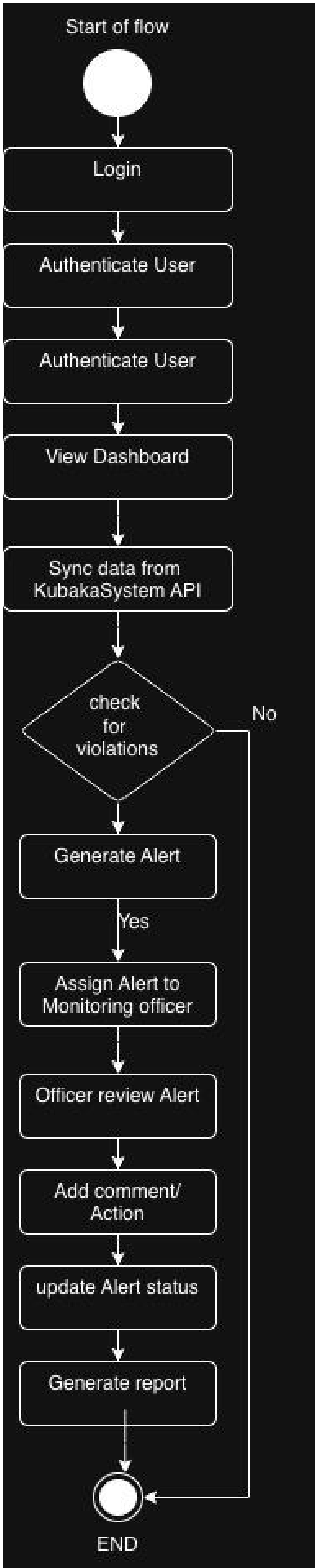
Start of flow
Login
Authenticate User
Authenticate User
View Dashboard
Sync data from KubakaSystem API
check for violations
No
Generate Alert
Yes
Assign Alert to Monitoring officer
Officer review Alert
Add comment/ Action
update Alert status
Generate report
END

Figure 7.4: Activity diagram

I. **Purpose**

The activity diagram describes the workflow followed by the system from user authentication to violation detection and report generation.

II. **Description**

The process begins when a user logs into the system and is authenticated successfully.

The system then:

- Displays the dashboard.
- Synchronizes project information from the Kubaka System API.
- Checks projects for violations.
- Determines whether a violation exists.

If no violation is found:

- The process ends.

If a violation is detected:

- An alert is generated.
- An alert is generated.
- The alert is assigned to a monitoring officer.
- The monitoring officer reviews the alert.
- Comments and corrective actions are recorded.
- The alert status is updated.
- A report is generated.

III. **Contribution**

The activity diagram illustrates the sequence of operations required to identify and manage project compliance issues, ensuring that violations are tracked and resolved efficiently.